

CSc 8830 — Computer Vision • Module 3

Image Blurring by Filtering: Spatial Domain vs. Fourier Domain

Student: Siri Bikkasani · **Institution:** Georgia State University

Submission: working web application + source code + this report + screen recording

GitHub repository → <https://github.com/siri423/Module3-ImageFiltering>

1. Objective

This assignment has two parts:

- **Implement** — image blurring using a *filtering* approach (convolving the image with a blur kernel), written from scratch.
- **Theory** — show that the result of a *spatial* filter equals the result of its *Fourier-domain* equivalent; i.e. that **convolution in space equals multiplication in frequency**, using the implementation above as experimental evidence.

Everything is delivered as a single **Streamlit web application** (`app.py`) in which every part is reachable through on-screen tabs, backed by a small, commented Python package in `src/`.

2. Implementation — blurring by filtering

Blurring an image means replacing every pixel with a weighted average of its neighbours. The pattern of weights is a small grid called a **kernel** (or mask). Sliding that kernel over the image and taking the weighted sum at each location is exactly a 2-D **convolution**:

$$(f * g)(x, y) = \sum_s \sum_t f(s, t) g(x - s, y - t)$$

where f is the image and g is the kernel. Two kernels are implemented (both normalised so their weights sum to 1, which keeps overall brightness unchanged):

- **Box (averaging) kernel** — every weight equal; a plain neighbourhood average.
- **Gaussian kernel** — weights follow a bell curve, giving a smoother, more natural blur:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2 + y^2)/(2\sigma^2)}$$

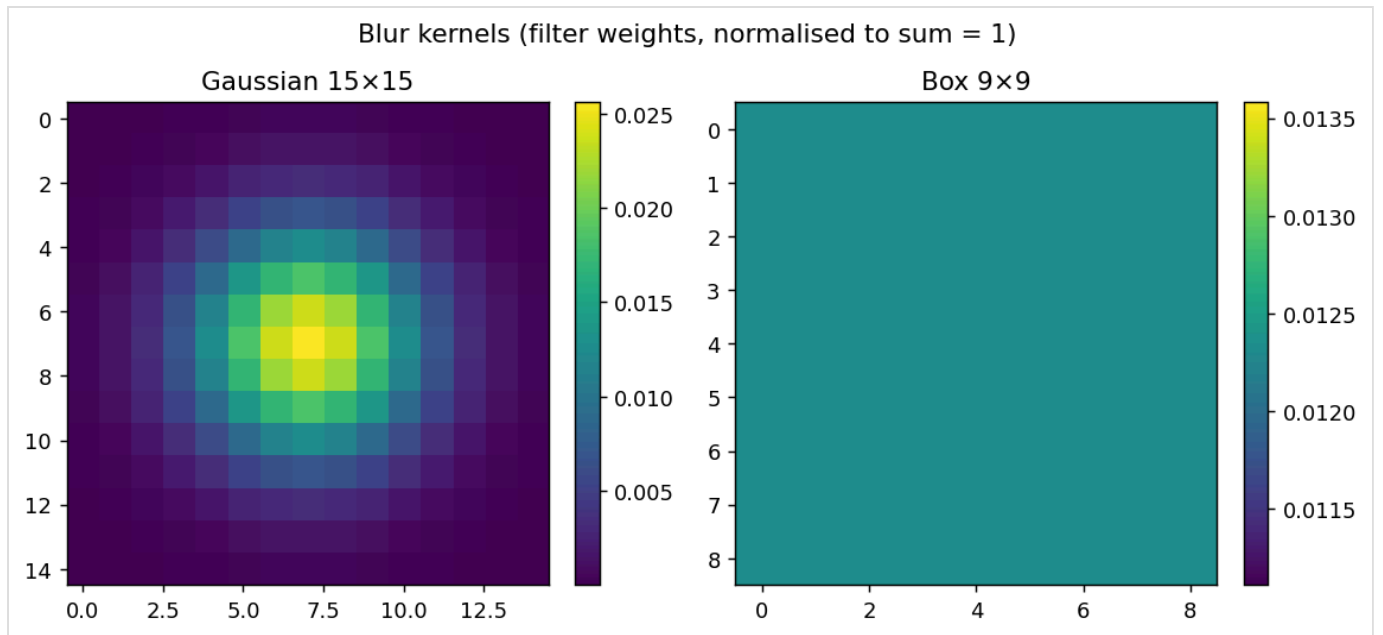


Figure 1 — the two blur kernels (filter weights). The Gaussian concentrates weight at the centre; the box spreads it evenly.

2.1 Spatial convolution, from scratch

The convolution is implemented in `src/spatial.py` without using any library blur function. A readable nested-loop version (`convolve2d_naive`) states the definition directly; a fast, vectorised “shift-and-add” version (`convolve2d`) gives an *identical* result by looping over the few kernel positions instead of over every pixel:

```
for i in range(kh):
    for j in range(kw):
        # padded[i:i+H, j:j+W] is the image shifted by (i, j)
        out += kflip[i, j] * padded[i:i+H, j:j+W]
```

The kernel is flipped (`kflip`) so this is true convolution rather than correlation; for symmetric blur kernels the two coincide. Image borders are handled by zero-padding (used for the proof, because it makes the operation a *linear* convolution) or edge-reflection (a nicer-looking visual blur).

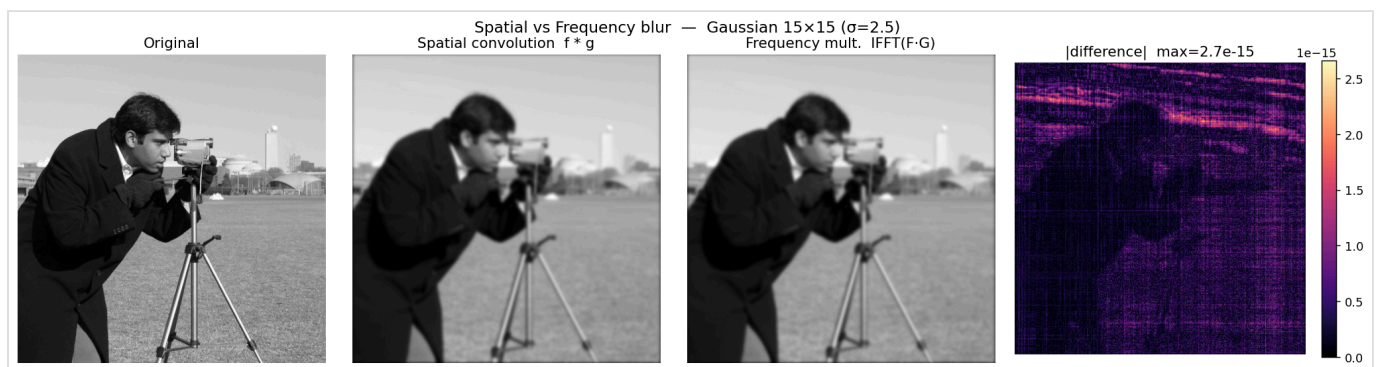


Figure 2 — the cameraman test image blurred with a Gaussian 15×15 kernel. Panels 2 and 3 are the two methods of Section 3; panel 4 is discussed there.

3. Theory — the Convolution Theorem

The 2-D **Discrete Fourier Transform (DFT)** of an $M \times N$ image is

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(ux/M + vy/N)}$$

The **Convolution Theorem** states that convolution in one domain corresponds to element-wise multiplication in the other:

$$f * g \Leftrightarrow F \cdot G$$

Consequently the blur can be computed *either* by sliding a kernel (Section 2) *or* entirely in the frequency domain:

$$f * g = \mathcal{F}^{-1} \{ \mathcal{F}\{f\} \cdot \mathcal{F}\{g\} \}$$

This is implemented in `src/frequency.py`: FFT the image, FFT the kernel, multiply the two spectra element-wise, and inverse-FFT.

3.1 Proof (via the DFT)

It is enough to prove the 1-D case; the 2-D result follows because the DFT is separable (applied along rows then columns). Let $\omega = e^{-j2\pi/N}$ and let $y[n] = \sum_m f[m] g[(n-m) \bmod N]$ be the circular convolution. Its DFT is

$$Y[k] = \sum_{n=0}^{N-1} \left(\sum_{m=0}^{N-1} f[m] g[(n-m) \bmod N] \right) \omega^{nk}$$

Substitute $p = (n-m) \bmod N$, so $n = m + p$ and, since $\omega^N = 1$, $\omega^{nk} = \omega^{mk} \omega^{pk}$. The double sum factors:

$$Y[k] = \left(\sum_m f[m] \omega^{mk} \right) \left(\sum_p g[p] \omega^{pk} \right) = F[k] G[k]$$

Thus the transform of a convolution is the product of the transforms. ■ The reverse pairing — multiplication in space corresponds to convolution in frequency — follows by the same argument applied to the inverse transform.

3.2 Linear vs. circular convolution

The raw FFT gives *circular* convolution, in which the kernel wraps around the image edges. To reproduce the ordinary zero-padded (*linear*) convolution of Section 2, both arrays are zero-padded to size $(M+k-1) \times (N+k-1)$ before transforming (here k is the kernel width) and the centre is cropped afterwards. This detail is why the two methods agree to rounding error in Section 5 rather than merely “looking similar.”

3.3 Why a blur kernel blurs

A box or Gaussian kernel is a **low-pass filter**: its transform $G(u, v)$ is large near the origin (low frequencies) and small far away (high frequencies). Forming $F \cdot G$ therefore keeps the smooth, slowly-varying content and suppresses fine detail and sharp edges — which is precisely what “blur” means.

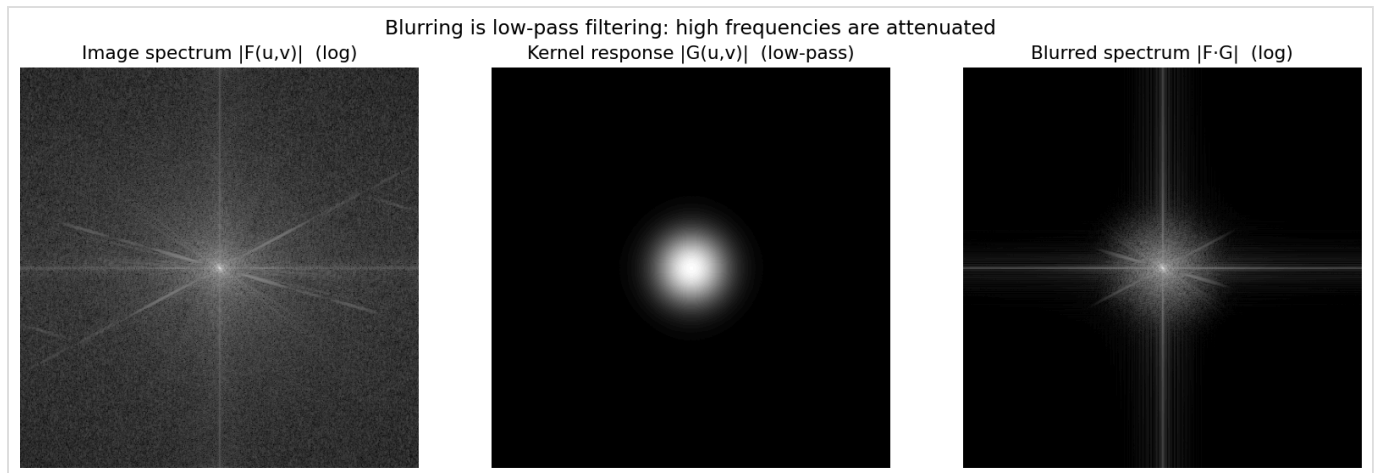


Figure 3 — frequency-domain view (log-magnitude spectra). Left: the image spectrum. Middle: the kernel’s response, bright in the centre (a low-pass filter). Right: the blurred spectrum, with high frequencies clearly attenuated.

4. Worked example — by hand

To make the theorem concrete, here is a small case worked out fully by hand and then confirmed by the code. Take the length-4 signal and 2-tap filter

$$f = [1, 2, 0, 3], \quad g = [1, 1, 0, 0]$$

(a) **Direct circular convolution.** With $y[n] = f[n] + f[n-1]$ (indices mod 4):

$$y = [1+3, 2+1, 0+2, 3+0] = [4, 3, 2, 3]$$

(b) **Via the DFT** (using $\omega = e^{-j2\pi/4} = -j$, so the powers of $\omega = e^{-j2\pi/N}$ are $1, -j, -1, j$):

$$F = [6, 1+j, -4, 1-j], \quad G = [2, 1-j, 0, 1+j]$$

Multiplying element-wise:

$$F \cdot G = [6 \cdot 2, (1+j)(1-j), (-4)(0), (1-j)(1+j)] = [12, 2, 0, 2]$$

The inverse DFT of $[12, 2, 0, 2]$ is

$$y = \frac{1}{4} \mathcal{F}^{-1}\{F \cdot G\} = [4, 3, 2, 3]$$

which is **identical** to the direct convolution in part (a). This is the theorem in miniature: multiplying the two transforms and inverting reproduces the convolution exactly.

2-D sanity check. Convolving the 3×3 patch $\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix}$ with a normalised 3×3 box kernel gives the centre output $\frac{1}{9}(10+20+\dots+90) = 50$, i.e. exactly the neighbourhood average — confirming the spatial-domain implementation as well.

5. Experimental validation

Method. The cameraman image is blurred with the *same* kernel two ways: (a) the from-scratch spatial convolution of Section 2, and (b) the Fourier-domain multiplication of Section 3. The two outputs are then compared pixel-by-pixel. If the theorem holds they must be equal up to floating-point rounding.

Panels 2–3 of Figure 2 are the two results; panel 4 is their absolute difference, whose largest value is $\sim 10^{-15}$ — the scale of machine epsilon, i.e. the images are the same. Repeating across kernels and sizes gives:

Kernel	Size	max difference	MSE	PSNR (dB)
gaussian	3×3	2.00e-15	2.14e-31	306.7
gaussian	7×7	1.22e-15	7.11e-32	311.5
gaussian	15×15	2.66e-15	2.23e-31	306.5
gaussian	25×25	3.89e-15	4.02e-31	304.0
box	3×3	1.78e-15	1.63e-31	307.9
box	7×7	1.78e-15	1.03e-31	309.9
box	15×15	6.99e-15	1.66e-30	297.8
box	25×25	1.53e-14	8.31e-30	290.8

Result: across every setting the maximum pixel disagreement is between 10^{-15} and 10^{-14} , and the PSNR is ~ 290 – 312 dB. Spatial filtering and its Fourier-domain equivalent produce the **same** image — convolution in space equals multiplication in frequency, as claimed. ■

The tiny residual is not error in either method; it is the accumulation of 64-bit floating-point rounding, and it grows gently with kernel size (more terms summed), exactly as expected.

6. Repository, and how to run

```
Module3_ImageFiltering/
├─ app.py           # Streamlit web app – all parts accessible here
├─ requirements.txt
├─ README.md
├─ src/
│   ├─ kernels.py   # box & Gaussian kernels
│   ├─ spatial.py    # from-scratch 2-D convolution (the blur)
│   ├─ frequency.py  # FFT-based filtering (multiply in frequency)
│   ├─ compare.py    # spatial-vs-frequency experiment + figures
│   └─ utils.py      # image load/save helpers
```

```
|— data/sample.jpg      # built-in test image
|— outputs/             # result figures used in this report
```

Run the web application (this is what the screen recording shows):

```
pip install -r requirements.txt
streamlit run app.py          # then open http://localhost:8501
```

Reproduce the experiment / figures from the command line:

```
python -m src.compare --kind gaussian --size 15
python -m src.compare --kind box --size 9
```

Where each requirement is demonstrated: blurring implementation → app tab “**1. Blur an image**”; spatial = frequency proof → app tab “**2. Spatial = Frequency**”; theory → app tab “**3. Theory**” and Sections 3–4 above.

7. References

- R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. — Ch. 3 (spatial filtering) and Ch. 4 (frequency-domain filtering; the convolution theorem).
- A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing* — DFT and circular convolution.
- NumPy FFT documentation, `numpy.fft`.